

Deadlock Simulation (No prevention)

Deadlock Simulator

Detection → Resolution techniques

1. Deadlock Scenario2. Dynamic Priority3. Victim Selection4. Wait-Die

Resource Allocation Graph

Four processes contend for three resources. P1, P2, and P3 form a circular wait — a deadlock. P4 is blocked waiting on R2 but is not part of the cycle.

GRAPH

```
graph TD
    P1((P1)) --> R1[R1]
    P1 --> R2[R2]
    P2((P2)) --> R2
    P2 --> R3[R3]
    P3((P3)) --> R3
    P3 --> R1
    P4((P4)) --> R2
```

Deadlock cycle edge

Non-cycle edge

⚠ Deadlock detected — circular wait: P1 → R2 → P2 → R3 → P3 → R1 → P1

PROCESSES

P1, P2, P3, P4

RESOURCES

R1, R2, R3

CYCLE LENGTH

3 processes

BLOCKED OUTSIDE CYCLE

P4 (wants R2)

ALLOCATION STATE

Process	Holds	Waiting for	Status
P1	R1	R2	Deadlocked
P2	R2	R3	Deadlocked
P3	R3	R1	Deadlocked
P4	—	R2	Blocked

NECESSARY CONDITIONS FOR DEADLOCK (ALL FOUR PRESENT)

Mutual exclusion — R1, R2, R3 are each held exclusively by one process

Hold and wait — every process in the cycle holds one resource while requesting another

No preemption — resources are not forcibly taken away from holders

Circular wait — P1 → P2 → P3 → P1 forms a closed cycle in the graph

Prevention Technique 1: Dynamic Priority Allocation

Deadlock Simulator

Detection → Resolution techniques

1. Deadlock Scenario2. Dynamic Priority3. Victim Selection4. Wait-Die

Dynamic Priority Allocation

Processes are assigned priority scores that change over time. A process that has waited longer gets an aging bonus, preventing starvation and letting the scheduler break the deadlock by always granting resources to the highest current priority.

Step 1 — Assign dynamic priorities — Each process starts with a base priority influenced by resource demand and estimated remaining work.

PRIORITY TABLE (LIVE SCORES)

Process	Base priority	Age bonus	Final score	Decision
P1	5	+2 (waiting 40ms)	7	Pending
P2	3	+1 (waiting 20ms)	4	Pending
P3	4	+1 (waiting 25ms)	5	Pending
P4	8	+0	8	Pending

← Previous step

Next step →

Deadlock Simulator

Detection → Resolution techniques

1. Deadlock Scenario2. Dynamic Priority3. Victim Selection4. Wait-Die

Dynamic Priority Allocation

Processes are assigned priority scores that change over time. A process that has waited longer gets an aging bonus, preventing starvation and letting the scheduler break the deadlock by always granting resources to the highest current priority.

Step 2 — Apply aging on contention — Processes waiting the longest receive an aging bonus added to their base priority, so no process starves indefinitely.

PRIORITY TABLE (LIVE SCORES)

Process	Base priority	Age bonus	Final score	Decision
P1	5	+2 (waiting 40ms)	7	Pending
P2	3	+1 (waiting 20ms)	4	Pending
P3	4	+1 (waiting 25ms)	5	Pending
P4	8	+0	8	Pending

← Previous step

Next step →

Deadlock Simulator

Detection → Resolution techniques

1. Deadlock Scenario2. Dynamic Priority3. Victim Selection4. Wait-Die

Dynamic Priority Allocation

Processes are assigned priority scores that change over time. A process that has waited longer gets an aging bonus, preventing starvation and letting the scheduler break the deadlock by always granting resources to the highest current priority.

Step 3 — Schedule highest priority — The scheduler grants the contended resource to the highest-scoring process; lower scoring processes are temporarily suspended, which breaks the cycle.

PRIORITY TABLE (LIVE SCORES)

Process	Base priority	Age bonus	Final score	Decision
P1	5	+2 (waiting 40ms)	7	Granted R2
P2	3	+1 (waiting 20ms)	4	Suspended
P3	4	+1 (waiting 25ms)	5	Suspended
P4	8	+0	8	Runs first

✓ Deadlock broken — P4 (priority 8) runs first and never touches the cycle, then P1 (priority 7, boosted) proceeds, releasing the rest of the chain.

← Previous step

Restart

Prevention Technique 2: Victim Selection

Deadlock Simulator

Detection → Resolution techniques

1. Deadlock Scenario2. Dynamic Priority3. Victim Selection4. Wait-Die

Victim Selection

Among the processes caught in the deadlock cycle, the resolver computes a weighted cost-to-abort score and aborts whichever process is cheapest to roll back. Lower score = cheaper to sacrifice.

VICTIM SELECTION CRITERIA

Criterion	Weight	P1	P2	P3
Resources held	30%	1	2	1
CPU time used	25%	60ms	20ms	45ms
Priority level	25%	5 (med)	3 (low)	4 (med)
Rollback cost	20%	Medium	Low	High

P1 cost score

6.4

higher = costlier

P2 cost score

3.1

higher = costlier

P3 cost score

7.8

higher = costlier

Run victim selection →

Deadlock Simulator

Detection → Resolution techniques

1. Deadlock Scenario

2. Dynamic Priority

3. Victim Selection

4. Wait-Die

Victim Selection

Among the processes caught in the deadlock cycle, the resolver computes a weighted cost-to-abort score and aborts whichever process is cheapest to roll back. Lower score = cheaper to sacrifice.

VICTIM SELECTION CRITERIA

Criterion	Weight	P1	P2	P3
Resources held	30%	1	2	1
CPU time used	25%	60ms	20ms	45ms
Priority level	25%	5 (med)	3 (low)	4 (med)
Rollback cost	20%	Medium	Low	High

P1 cost score

6.4

higher = costlier

P2 cost score

3.1

← lowest cost

P3 cost score

7.8

higher = costlier

Victim selected: P2 — lowest cost to abort and roll back

RESOLUTION SEQUENCE

- P2 selected as victim (cost score 3.1)
- P2 aborted — releasing held resources
- Free'd resource now available to its waiter
- Next process in the former cycle can proceed
- P4 unblocked once its required resource is free
- Cycle dissolved. System resumes normal operation.

Prevention Technique 3: Wait-Die

Deadlock Simulator

Detection → Resolution techniques

1. Deadlock Scenario

2. Dynamic Priority

3. Victim Selection

4. Wait-Die

Wait-Die Scheme

WAIT (older requests from younger)

If the requesting process is older than the holder, it waits patiently.

DIE (younger requests from older)

If the requesting process is younger, it is aborted and restarts with its original timestamp.

LIVE SIMULATION — SCENARIO 1 OF 3

Scenario 1 — older process requests a resource held by a younger one

P1 (timestamp 50, older) requests R2, held by P2 (timestamp 20, younger)

Decision

P1 is older than P2 -> P1 WAITS for P2 to release R2

Old requesting from young is allowed to wait. This is safe: the older process can never end up indirectly waiting on something the younger one needs, because of how timestamps order requests.

TIMESTAMP ORDER (LOWER = OLDER = ARRIVED FIRST)

Process	Timestamp	Role this scenario	Action
P1	T=50ms	Requester	Wait
P2	T=20ms	Holder	Holds resource
P3	T=15ms	—	Not involved
P4	T=5ms	—	Not involved

← Previous scenario

Next scenario →

Deadlock Simulator

Detection → Resolution techniques

1. Deadlock Scenario2. Dynamic Priority3. Victim Selection4. Wait-Die

Wait-Die Scheme

WAIT (older requests from younger)

If the requesting process is older than the holder, it waits patiently.

DIE (younger requests from older)

If the requesting process is younger, it is aborted and restarts with its original timestamp.

LIVE SIMULATION — SCENARIO 2 OF 3

Scenario 2 — younger process requests a resource held by an older one
P3 (timestamp 15, younger) requests R1, held by P1 (timestamp 50, older)

Decision
P3 is younger than P1 -> P3 DIES (aborted, restarts with its original timestamp 15)

Young requesting from old must die. P3 is rolled back immediately. It restarts carrying its original timestamp, so it grows relatively "older" over time and is guaranteed to eventually win.

TIMESTAMP ORDER (LOWER = OLDER = ARRIVED FIRST)

Process	Timestamp	Role this scenario	Action
P1	T=50ms	Holder	Holds resource
P2	T=20ms	—	Not involved
P3	T=15ms	Requester	Die -> restart
P4	T=5ms	—	Not involved

← Previous scenario

Next scenario →

Deadlock Simulator

Detection → Resolution techniques

1. Deadlock Scenario2. Dynamic Priority3. Victim Selection4. Wait-Die

Wait-Die Scheme

WAIT (older requests from younger)

If the requesting process is older than the holder, it waits patiently.

DIE (younger requests from older)

If the requesting process is younger, it is aborted and restarts with its original timestamp.

LIVE SIMULATION — SCENARIO 3 OF 3

Scenario 3 — newest process requests a resource held by an older one
P4 (timestamp 5, newest) requests R2, held by P2 (timestamp 20, older)

Decision
P4 is younger than P2 -> P4 DIES, re-queued keeping timestamp 5

Wait-Die prevents circular wait by construction: a younger process never holds a resource while waiting on an older one, so a cycle made entirely of "younger waits on older" edges can never close.

TIMESTAMP ORDER (LOWER = OLDER = ARRIVED FIRST)

Process	Timestamp	Role this scenario	Action
P1	T=50ms	—	Not involved
P2	T=20ms	Holder	Holds resource
P3	T=15ms	—	Not involved
P4	T=5ms	Requester	Die -> restart

✓ Wait-Die ensures no circular wait can ever form — deadlock prevention by design, not detection-and-recovery.

← Previous scenario

Restart